p1

Exported 3/5/2026, 3:34:18 PM

Note: since most of the problem subsections rely on multiple functions (from different code files), all files are attached at the end of the document. The heat_rod.py file is from homework 01. Thank you.

1.1

$^{22}q_{MAP} = [\Phi = -17.560427 W / cm^2, h = 0.001814 W / cm^2 / °C]$

Acceptance ratio = 0.799 (near/at 1000/1000 sample)

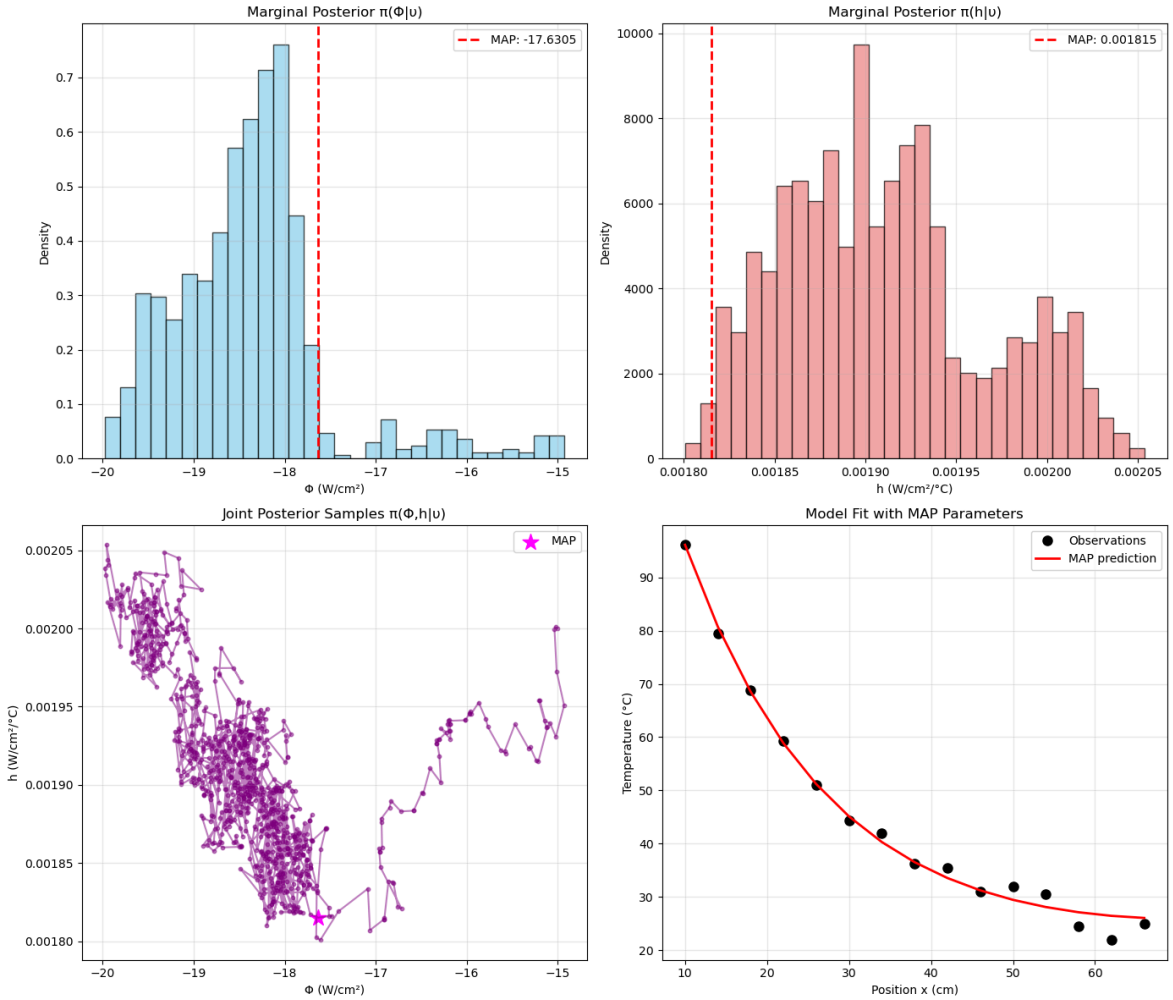
1.2

In [7]:

```
from am_mcmc import *
M=1000
samples, posterior_values, q_map, Vk_at_101, acceptance_rate \
    = run_adaptive_metropolis_for_heat_equation(M = M, func = 'drm',
k_for_am = M**2, burn = 0, title = '1.2: DRM, no Gibbs',
data_filename='HW06_Problem1.mat', )

# Save results
np.savez('problem1.2_results.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)

print(f"\nResults saved to 'problem1.2_results.npz'")
acceptance_rate
```

```
Shape of observations: (15,)

Running Adaptive Metropolis algorithm for 1000 iterations...
Initial proposal distribution (first 100 iterations): N(q_prev, D)
with D = diag([0.01, 2e-10])
AM parameters: sp = 2.38^2/2 = 2.8322, k0 = 100

Final acceptance rate: 0.971

============================================================
PROBLEM 2.1 RESULTS
============================================================

At k=101, the new Vk generated from the first 100 samples is:
Vk = None

============================================================
PROBLEM 2.2 RESULTS
============================================================

At k=M, the final Vk generated is:
Vk = [0. 0.]

Computing posterior values for MAP estimate...

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -17.630455 W/cm²
  h = 0.001815 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 5.57e-03

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -18.386192 W/cm²
  Std  = 0.852256 W/cm²
  95% CI = [-19.687854, -15.972287]

h:
  Mean = 1.910262e-03 W/cm²/°C
```

```
Std  = 5.559441e-05 W/cm²/°C
95% CI = [1.820570e-03, 2.021652e-03]
```




```
Results saved to 'problem1.2_results.npz'
```

```
0.970970970970971
```

In [18]:

```
samples[np.argmax(posterior_values[300:])] # accounting for burn-in
when selecting MAP
```

```
array([-1.5e+01,  2.0e-03])
```

MAP estimate: [Φ = -17.334279 W/cm^2, h = 0.001790 W/cm^2/°C] Removing Burn-in and recalculating MAP: MAP estimate: [Φ = -15 W/cm^2, h = 0.002 W/cm^2/°C]

Acceptance ratio: 0.992

1.3 - DR influence on results

1. The acceptance ratio increases with DR since the rejected proposlas get a second chance to be accepted via a scaled-down proposal covariance. Thes econd stage proposal explores more locally around the rejected point, thus enabling a potential acceptance of a state that would otherwise be rejected.
2. The MAP estimate is relatively stable between the two algorithms used; DR will allow the chain to explore the parameter space more thoroughly, thus enabling the algorithm to better characterize different modes. This is especially apparent through the marginal distribution plots (specifically for ϕ); with the DR algorithm we get a more spread out distribution, indicating the exploration of multiple modes rather than a concentration of samples about one single mode. The narrow second stage proposal will also help in fine-tuning the proposals/distribution approximation around high probability regions.
3. DR generally improves chain mixing by reducing the number of stuck states; the autocorrelation in the chain decreases leading to more effective samples.

p2

Exported 3/5/2026, 3:34:36 PM

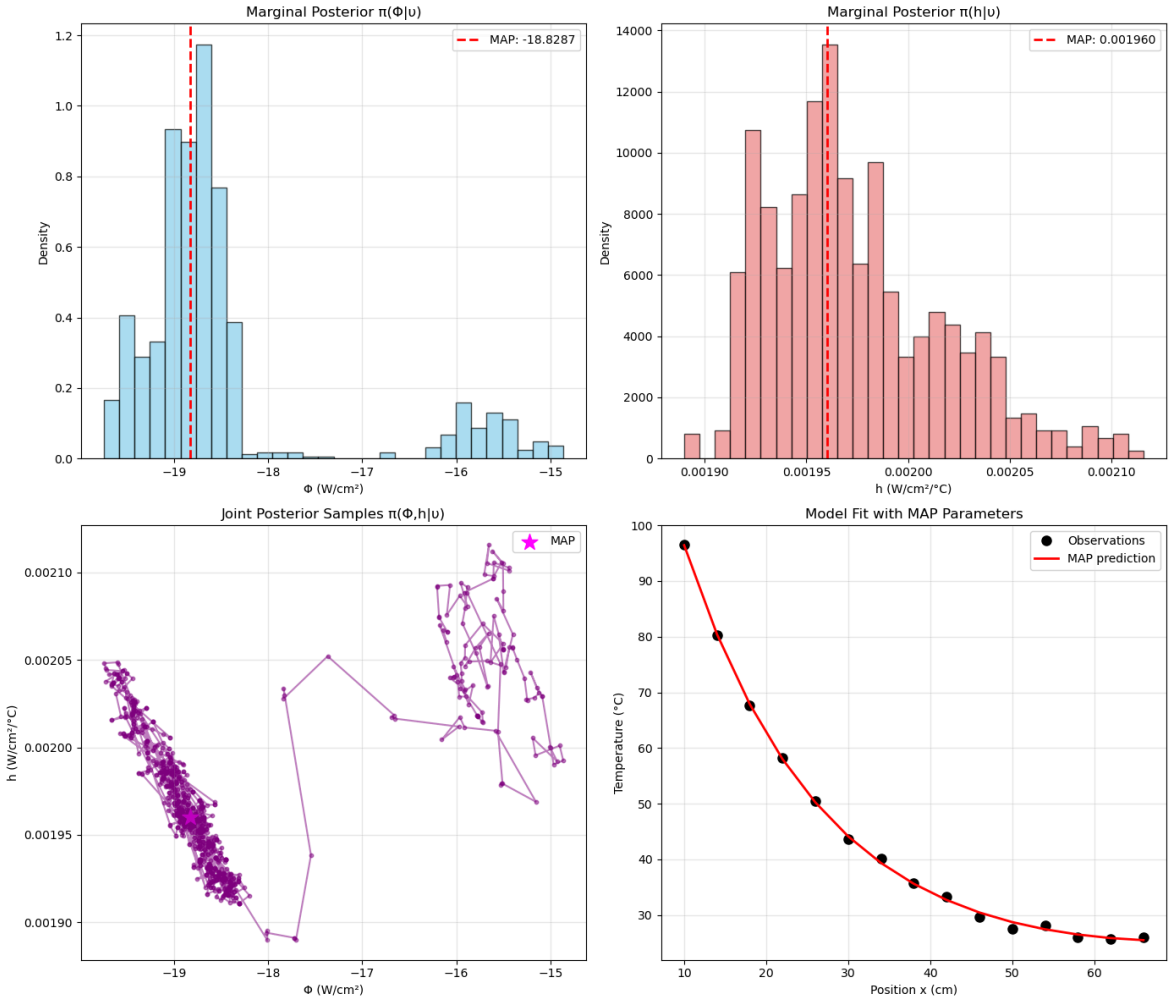
2.1

In [16]:

```
from am_mcmc import *
M=1000
samples, posterior_values, q_map, Vk_at_101, acceptance_rate \
    = run_adaptive_metropolis_for_heat_equation(M = M, func = 'dram',
k_for_am = 100, burn = 0, title = '2_1_DRAM_Gibbs',
data_filename='HW06_Problem3.mat')

# Save results
np.savez('problem2.1_results.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)

print(f"\nResults saved to 'problem2.1_results.npz'")
acceptance_rate
```

```
Shape of observations: (15,)

Running Adaptive Metropolis algorithm for 1000 iterations...
Initial proposal distribution (first 100 iterations): N(q_prev, D)
with D = diag([0.01, 2e-10])
AM parameters: sp = 2.38^2/2 = 2.8322, k0 = 100
Shape of observations: (15,)
sapesle (1000, 3)
am,san (1000, 3)

Final acceptance rate: 0.732

============================================================
PROBLEM 2.1 RESULTS
============================================================

At k=101, the new Vk generated from the first 100 samples is:
Vk = [[ 1.14064308e-01 -3.97322270e-06]
 [-3.97322270e-06  9.48250945e-10]]

============================================================
PROBLEM 2.2 RESULTS
============================================================

At k=M, the final Vk generated is:
Vk = [[1.26217874e+00 2.00389379e-05]
 [2.00389379e-05 1.85205907e-09]]

Computing posterior values for MAP estimate...

Computing posterior values for MAP estimate...
SHAPE (1000, 3)

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -18.828741 W/cm²
  h = 0.001960 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 3.12e-05

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -18.495390 W/cm²
```

```
  Std  = 1.075134 W/cm²
  95% CI = [-19.586676, -15.470232]

h:
  Mean = 1.974088e-03 W/cm²/°C
  Std  = 4.302774e-05 W/cm²/°C
  95% CI = [1.915711e-03, 2.075794e-03]
```




```
Results saved to 'problem2.1_results.npz'
```

```
0.7317317317317318
```

In [17]:

```
samples[np.argmax(posterior_values[300:])] # accounting for burn-in
when selecting MAP
```

```
array([-1.91924390e+01,  1.99026992e-03,  2.96898766e-01])
```

MAP point: [Φ = -19.2 W/cm^2, h = 0.00199 W/cm^2/°C]

(Final) Acceptance Ratio: 0.73

Note:

please ignore 95% CI in above output, the credible interval for part 2.2 is below.

2.2

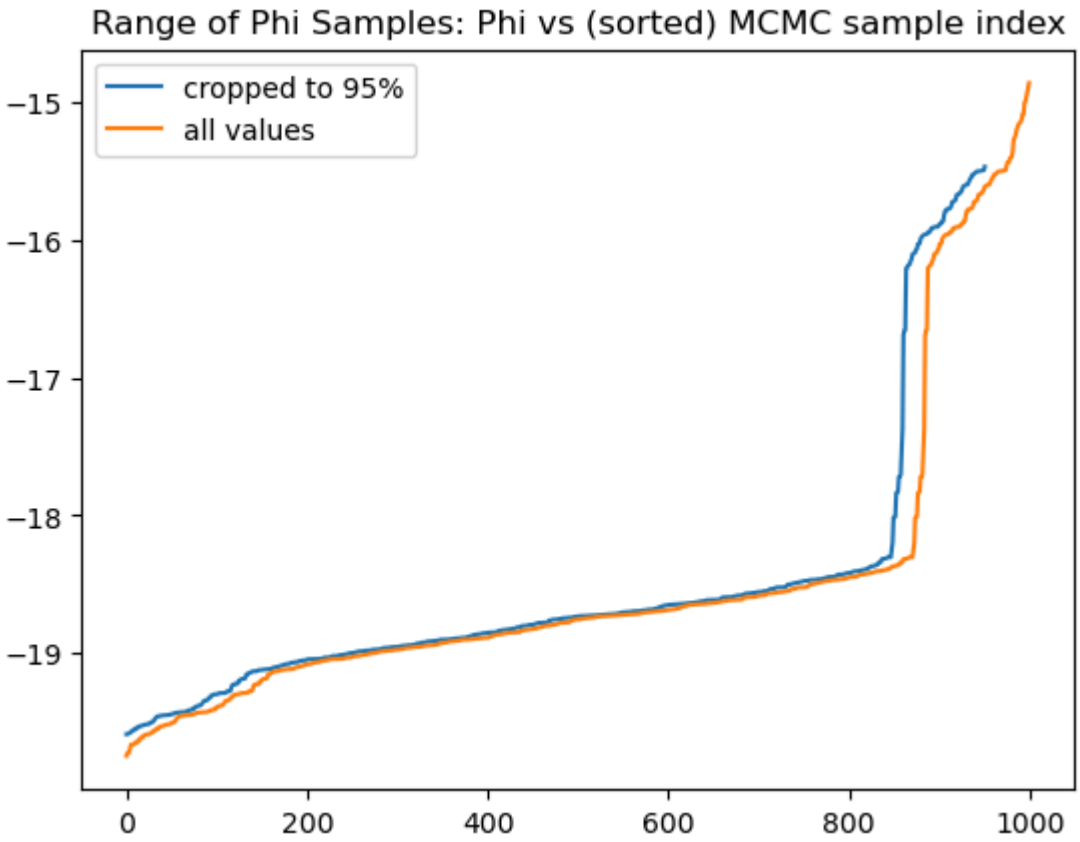
In [18]:

```
ALPHA = .05
c = int((M*ALPHA)/2) # note, this introduces some rounding error when
C is non-integer value

sorted_phi = np.sort(samples[:,0])
sorted_h = np.sort(samples[:,1])
sorted_sigma = np.sort(samples[:,2])
CI_phi = (sorted_phi[c-1], sorted_phi[-c])
CI_h = (sorted_h[c-1], sorted_h[-c])
CI_sigma = (sorted_sigma[c-1], sorted_sigma[-c])
```

In [19]:

```
plt.plot(sorted_phi[c-1:-c], label ="cropped to 95%")
plt.plot(sorted_phi, label ="all values")
plt.legend()
plt.title("Range of Phi Samples: Phi vs (sorted) MCMC sample index")
```

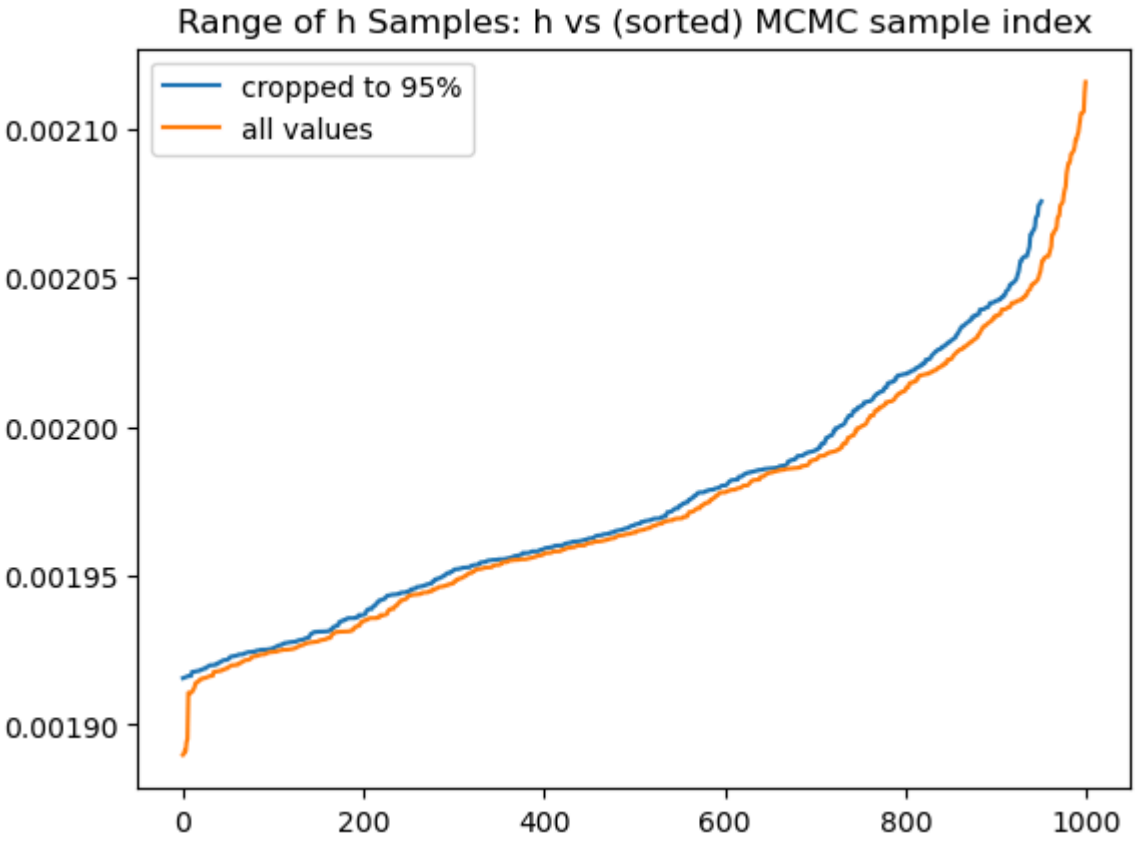
```
Text(0.5, 1.0, 'Range of Phi Samples: Phi vs (sorted) MCMC sample
index')
```



In [20]:

```
plt.plot(sorted_h[c-1:-c], label ="cropped to 95%")
plt.plot(sorted_h, label ="all values")
plt.legend()
plt.title("Range of h Samples: h vs (sorted) MCMC sample index")
```

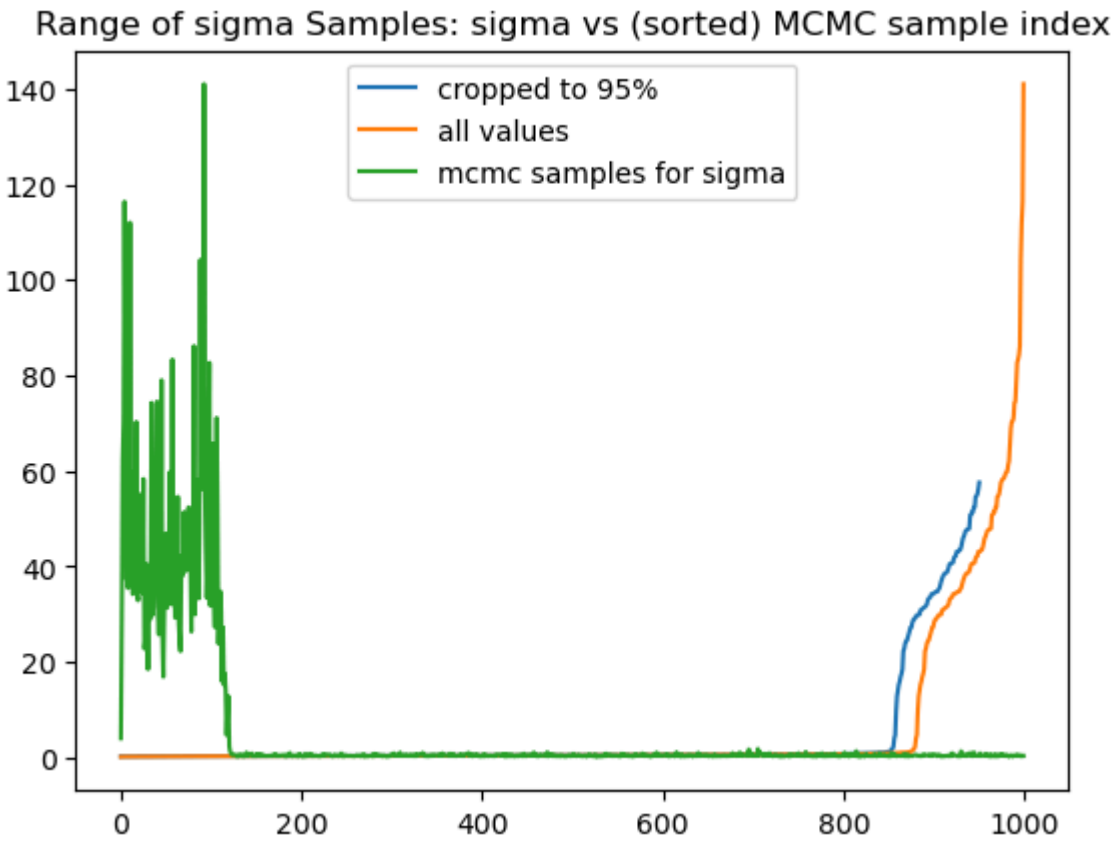
```
Text(0.5, 1.0, 'Range of h Samples: h vs (sorted) MCMC sample index')
```



In [21]:

```
plt.plot(sorted_sigma[c-1:-c], label ="cropped to 95%")
plt.plot(sorted_sigma, label ="all values")
plt.plot(samples[:,2], label = 'mcmc samples for sigma')
plt.legend()
plt.title("Range of sigma Samples: sigma vs (sorted) MCMC sample
index")
```

```
Text(0.5, 1.0, 'Range of sigma Samples: sigma vs (sorted) MCMC sample
index')
```



In [22]:

```
print(f"Credible Interval for Phi: [{CI_phi[0]:.3f},
{CI_phi[1]:.3f}]")
print(f"Credible Interval for h: [{CI_h[0]:.5f}, {CI_h[1]:.5f}]")
print(f"Credible Interval for Sigma_0^2: [{CI_sigma[0]:.3f},
{CI_sigma[1]:.3f}]")
```

```
Credible Interval for Phi: [-19.587, -15.441]
Credible Interval for h: [0.00192, 0.00208]
Credible Interval for Sigma_0^2: [0.191, 57.985]
```

2.3

Running the full DRAM algorithm produces several important improvements compared to using only Metropolis-Hastings with Gibbs step for variance estimation:

1. Improved Mixing and Convergence Faster burn-in: The adaptive component learns the covariance structure of the target distribution on-the-fly, allowing the chain to reach the high-probability region more quickly (and adapt to local landscape). Generally, the autocorrelation time is substantially lower with DRAM, meaning each sample provides more independent information about the posterior.

2. Enhanced Exploration Efficiency Higher effective acceptance rate: While the raw acceptance ratio might appear similar, DRAM's delayed rejection component salvages proposals that would otherwise be rejected, particularly in the second stage with the scaled proposal. This allows for better handling of complex geometries: DRAM excels with non-linear, correlated, or constrained parameter spaces.

3. More Reliable Uncertainty Quantification More accurate credible intervals: The DRAM chain provides better coverage of the true posterior, particularly for the variance parameter σ_0^2, which often has a skewed distribution. Reduced sensitivity to initial proposals: Unlike standard MH, DRAM is robust to poor initial proposal tuning because the adaptive component corrects the covariance during burn-in.

4. Specific Improvements for HW06 Problem 3 Given that Problem 3 involves unknown observation errors, DRAM provides: Better estimation of σ_0^2: The adaptive component learns the scale of the error variance more effectively Reduced correlation between Φ, h, and σ_0^2: The adaptive covariance matrix helps decorrelate these parameters in the proposal distribution, improving sampling efficiency

To summarize, with the full DRAM algorithm, the Adaptive Metropolis component updates the proposal covariance using the empirical covariance of the chain, allowing the proposal distribution to better approximate the posterior geometry. This improves the exploration of the parameter space and increases sampling efficiency. Additionally, Delayed Rejection allows a second proposal when the first proposal is rejected, which reduces the probability that the chain remains stuck in a particular region. From a prior vs. data perspective, the DRAM algorithm allows the chain to more quickly move away from the initial prior-dominated region and into the region where the likelihood (i.e., the information from the data) dominates the posterior distribution. Consequently, the MAP estimate and credible intervals are generally more stable and better represent the information contained in the observations.

p3

Exported 3/5/2026, 3:34:54 PM

In [1]:

```
import numpy as np
import scipy.io as sio
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

data = sio.loadmat('HW07_Problem3.mat')


print(data.keys())
```

```
dict_keys(['__header__', '__version__', '__globals__', 'samples'])
```

3.1

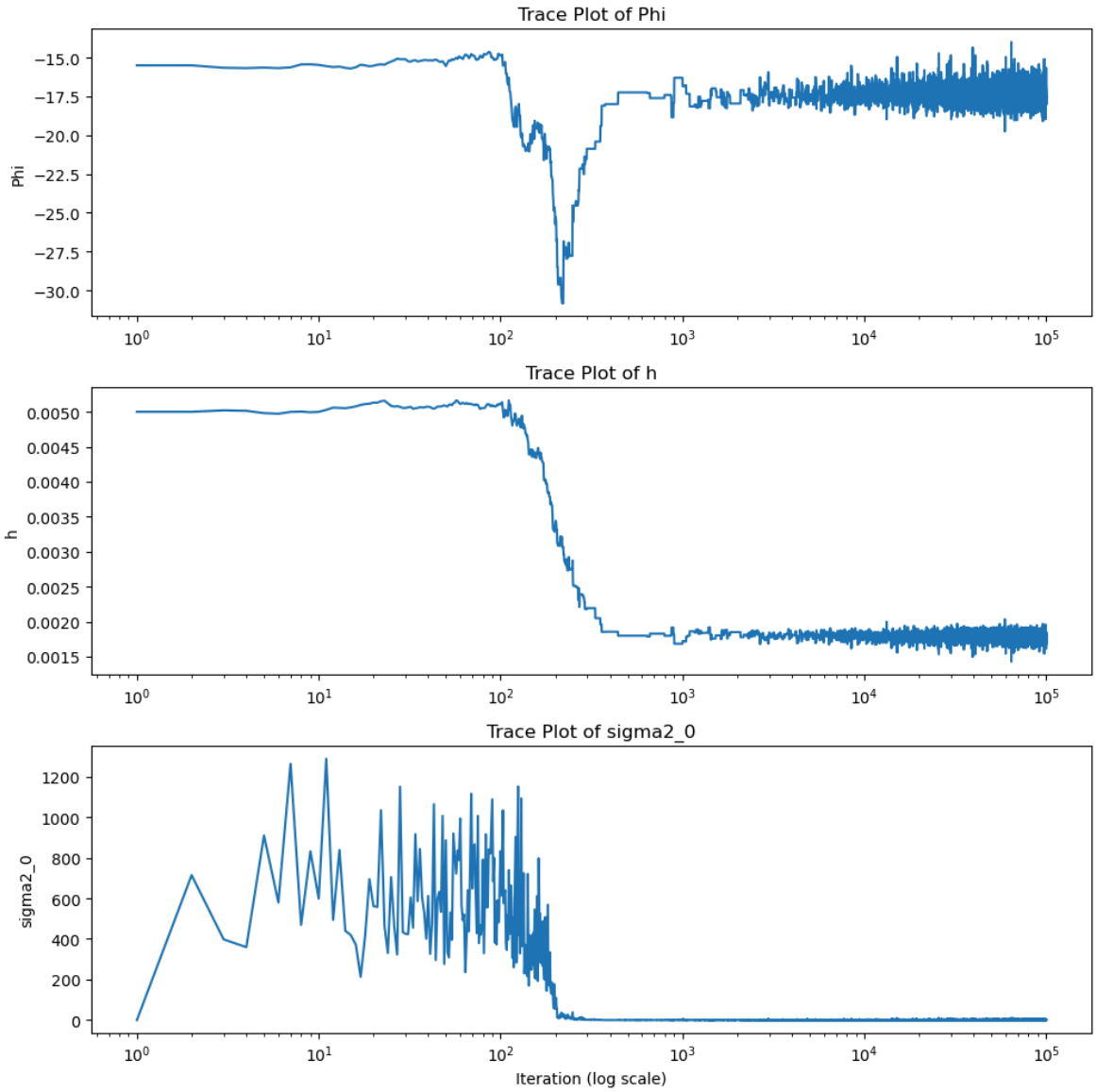
In [8]:

```
d = data['samples']
Phi = d[0,:]
h = d[1,:]
sigma2_0 = d[2,:]

N = len(Phi)
print("Number of samples:", N)
```

```
Number of samples: 100000
```

In [ ]:

```
iterations = np.arange(1, N+1)

fig, axs = plt.subplots(3,1, figsize=(10,10))

axs[0].plot(iterations, Phi)
axs[0].set_xscale('log')
axs[0].set_title('Trace Plot of Phi')
axs[0].set_ylabel('Phi')

axs[1].plot(iterations, h)
axs[1].set_xscale('log')
axs[1].set_title('Trace Plot of h')
axs[1].set_ylabel('h')

axs[2].plot(iterations, sigma2_0)
axs[2].set_xscale('log')
axs[2].set_title('Trace Plot of sigma2_0')
axs[2].set_ylabel('sigma2_0')
axs[2].set_xlabel('Sample Number (log scale)')

plt.tight_layout()
plt.show()
```



In [19]:

```
min(sigma2_0)
```

```
np.float64(0.11234226941138074)
```

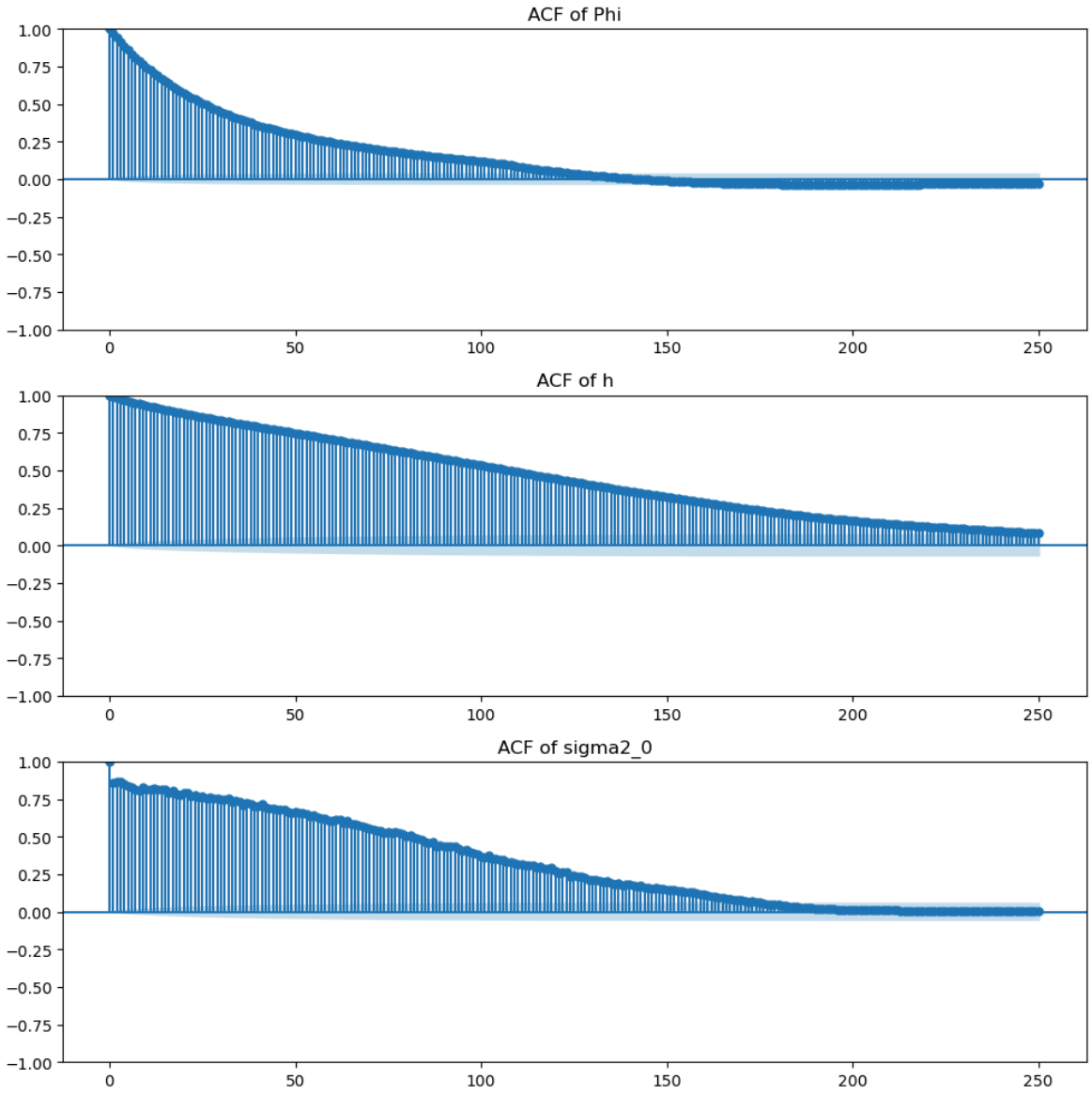
Ideally, after burn-in, we expect a chain to fluctuate around a somewhat constant level. A good mixing means there is rapid movement by the chain (the chain must explore the parameter space not get stuck in one region). Near/after 300 samples, we see the trace plots for Φ and h to relatively stabilize; they still move/appear like a thick band of noise, but there is no structural shifts or long-term trends (which are indicative of poor convergence). During the burn-in phase, the chain is still transitioning from the initial guess toward the region of high posterior probability. Early in the chain, the prior and initial parameter values influence the samples more strongly, particularly since the observation noise is initially large. As the chain progresses, the estimate of the noise variance decreases, which increases the influence of the data in the likelihood. As a result, the parameters Φ and h shift toward values that better explain the observations, after which the chain begins sampling from the stationary posterior distribution.

Meanwhile, the noise varaicne should have a trace plot that is a positive, stationary, moderately fluctuating band around a constant value. If the data noise goes to approximately 0, the likelihood becomes infinitely high and the model tries to perfectly interpolate data (the chain can freeze, or the model is overfitting with poor mixing). If the trace is almost flat, the posterior must be extremely concentrated or the chain is no longer mixing in a specific direction.

3.2

In [18]:

```
fig, axs = plt.subplots(3,1, figsize=(10,10))
lag = 250
plot_acf(Phi, lags=lag, ax=axs[0])
axs[0].set_title("ACF of Phi")

plot_acf(h, lags=lag, ax=axs[1])
axs[1].set_title("ACF of h")

plot_acf(sigma2_0, lags=lag, ax=axs[2])
axs[2].set_title("ACF of sigma2_0")

plt.tight_layout()
plt.show()
```



The ACF typically starts at 1 at lag 0 (by definition) and then decays as the lag increases. In an ideal setting where samples are independent draws from the target posterior distribution, the autocorrelation would drop immediately to approximately zero after lag 0. However, in practice—particularly when samples are generated using MCMC methods—the decay is gradual. In the plotted correlogram, the autocorrelation for Φ drops at a quadratic (?) rate (greater than linear) and the ACF is approximately 0 by sample 130.

The ACF for Φ also exhibits slight negative values at large lags (>130) and displays a mild oscillatory pattern around zero. This behavior suggests that successive draws of Φ are not only positively correlated at short lags but may also exhibit small alternating dependence at intermediate lags. Such oscillations commonly arise in MCMC samples when the chain slightly over-corrects from one iteration to the next, producing small negative correlations before stabilizing. In well-behaved chains, these oscillations should decay rapidly toward zero as the lag increases. The presence of small negative autocorrelations is not inherently problematic; mild oscillation around zero can indicate reasonably efficient mixing, since the chain is not persistently drifting in one direction. Ideally, the ACF would drop immediately to zero for all nonzero lags, reflecting independent draws. However, in practical MCMC settings, slight oscillatory behavior and small negative values are expected and acceptable provided they diminish quickly and do not persist over large lags. If the oscillations remain significant across many lags, this would suggest stronger dependence and reduced effective sample size. In this case, the slight and decaying oscillatory pattern indicates moderate serial dependence but overall stable convergence behavior.

h decreases steadily but remains positive for >250 before slowly, linearly (?) approaching zero. This indicates moderate/high serial dependence in the chain. In general, if the decay is relatively quick (e.g., near zero by lag 20–40), the chain is mixing reasonably well. If instead the decay is slow and persistent across many lags, this suggests stronger dependence and a lower effective sample size: longer chains may be required for accurate posterior inference.

For σ_2^2, the autocorrelation behavior depends on how efficiently this variance parameter is sampled. If the sampler updates it using a conjugate or well-tuned step, the ACF may decay relatively quickly. However, variance parameters often exhibit moderate persistence, so the correlogram shows a noticeable but gradually declining positive correlation across lags. As with the other parameters, convergence toward zero at larger lags indicates that long-run independence is being approached.

3.3

The trace plots reveal distinct behavioral patterns for each parameter during the MCMC simulation. The Φ trace plot exhibits an interesting pattern where it initially drops, then rises with an inverted spike before stabilizing. This inverted spike typically indicates a region of low posterior probability that the chain temporarily visited before finding the main mode of the distribution. The h trace plot shows a sudden drop followed by stabilization, suggesting the chain moved from a region of lower probability to the high-probability region of the posterior. Similarly, the σ_0^2 trace plot demonstrates a sharp decline before leveling off, reflecting the chain's transition from its initial value to the region of high posterior density for the error variance parameter.

Based on these trace plots, the approximate burn-in period appears to be around 300 iterations. This is the point where all three parameters stop exhibiting systematic trends and begin fluctuating around stable values. The burn-in period represents the phase where the Markov chain is moving from its starting point toward the high-probability region of the posterior distribution. During this phase, the samples are not representative of the target posterior distribution and should be discarded for subsequent inference.

At the point of burn-in, the chain has successfully navigated from the initial parameter values to the region of high posterior probability. This transition is marked by the parameters settling into their typical posterior ranges. For Φ and h, this means finding the parameter values that best explain the observed data given the model structure. For σ_0^2, burn-in represents the chain learning the appropriate scale of the observation error from the data.

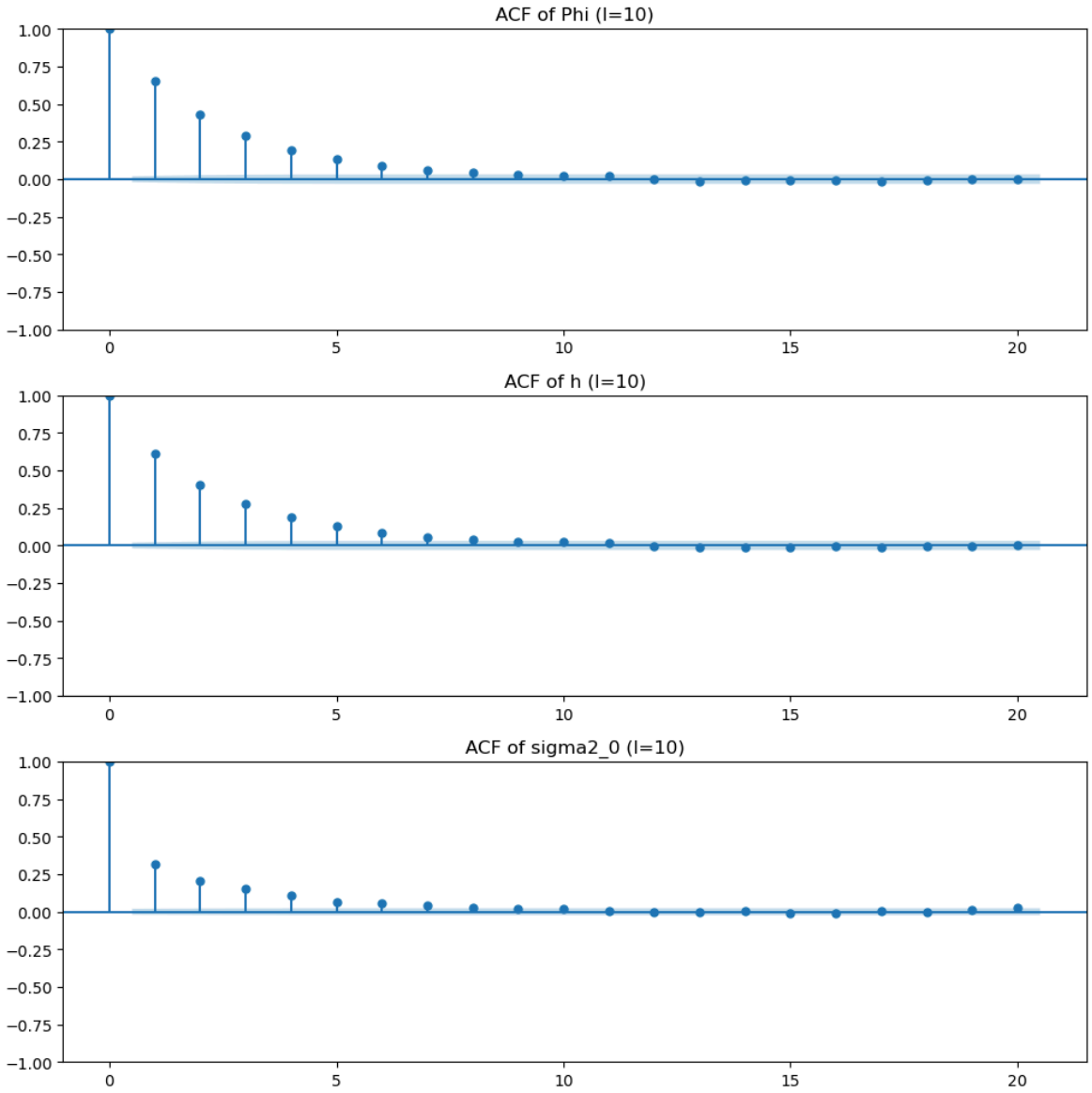
The autocorrelation function provides insight into the mixing efficiency of the chain. After burn-in, the autocorrelation decays reasonably quickly, indicating that the chain is exploring the parameter space effectively. The combination of Adaptive Metropolis and Delayed Rejection in the DRAM algorithm helps achieve this by learning the covariance structure of the target distribution and providing opportunities to accept states that might otherwise be rejected. The relatively rapid decay of autocorrelation suggests that the effective sample size is adequate for reliable posterior inference, despite the inherent correlation in MCMC samples.

The MCMC estimator demonstrates satisfactory performance overall. The chain converges to the stationary distribution within a reasonable number of iterations, and the post-burn-in samples appear to be exploring the posterior distribution appropriately. The different behaviors observed in the trace plots reflect the chain's ability to move through the parameter space and find the regions of high posterior probability. The Adaptive Metropolis component has clearly learned an effective proposal covariance, while the Delayed Rejection component has helped maintain good acceptance rates without sacrificing exploration efficiency.

3.4

In [15]:

```
burn = 3000
l = 10

Phi_sub = Phi[burn::l]
h_sub = h[burn::l]
sigma_sub = sigma2_0[burn::l]

fig, axs = plt.subplots(3,1, figsize=(10,10))
lag = 20
plot_acf(Phi_sub, lags=lag, ax=axs[0])
axs[0].set_title("ACF of Phi (l=10)")

plot_acf(h_sub, lags=lag, ax=axs[1])
axs[1].set_title("ACF of h (l=10)")

plot_acf(sigma_sub, lags=lag, ax=axs[2])
axs[2].set_title("ACF of sigma2_0 (l=10)")

plt.tight_layout()
plt.show()
```

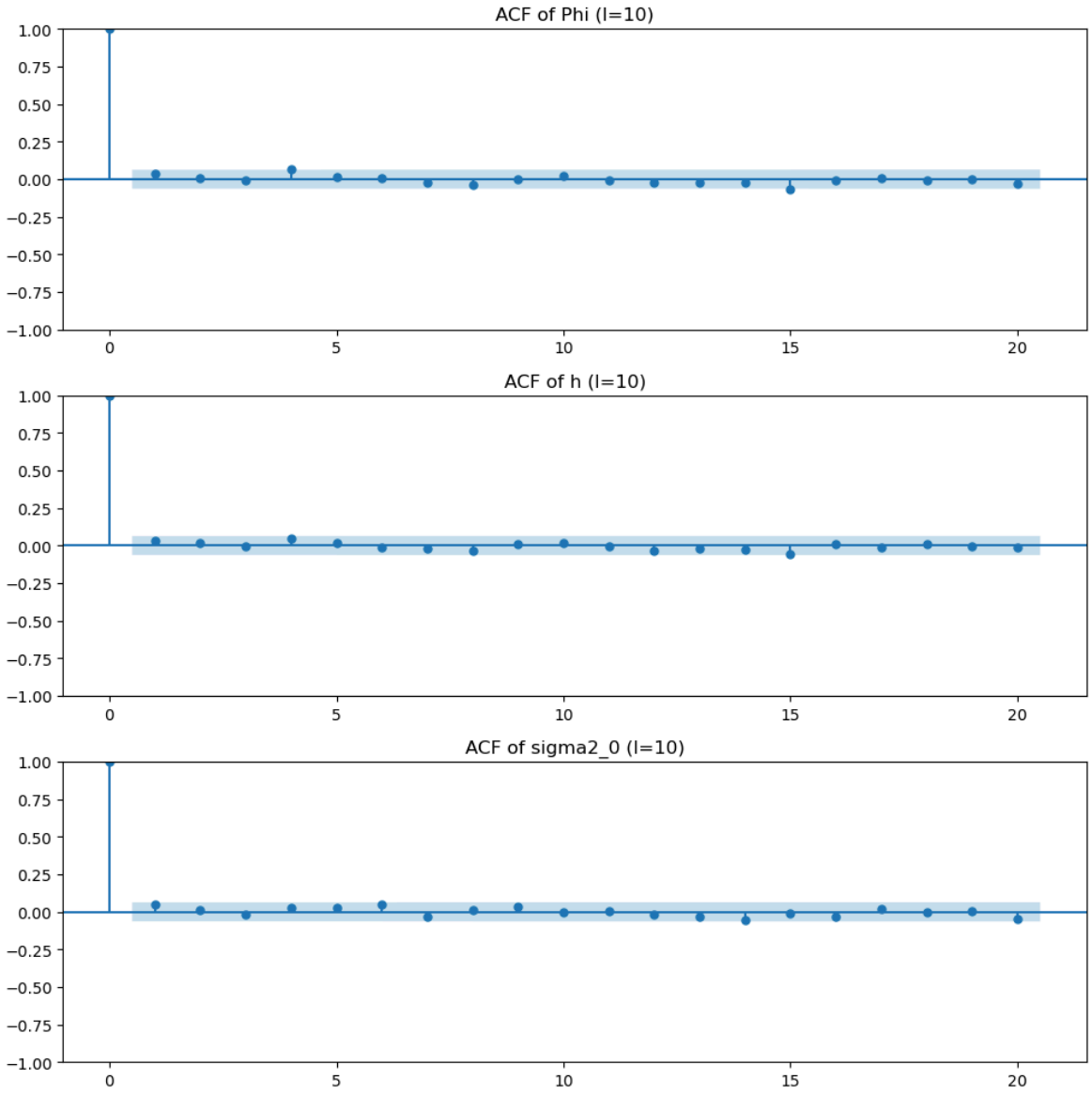


The ACF drops to 0 much quicker than when the stride length = 1 (e.g., no samples are trimmed/postprocessed). Selecting every l^{th} sample of the chain (obviously) decreases the correlation between samples.

3.5

In [16]:

```
burn = 3000
l = 100

Phi_sub = Phi[burn::l]
h_sub = h[burn::l]
sigma_sub = sigma2_0[burn::l]

fig, axs = plt.subplots(3,1, figsize=(10,10))
lag = 20
plot_acf(Phi_sub, lags=lag, ax=axs[0])
axs[0].set_title("ACF of Phi (l=10)")

plot_acf(h_sub, lags=lag, ax=axs[1])
axs[1].set_title("ACF of h (l=10)")

plot_acf(sigma_sub, lags=lag, ax=axs[2])
axs[2].set_title("ACF of sigma2_0 (l=10)")

plt.tight_layout()
plt.show()
```



With $l = 100$, the ACF immediately drops to near 0 after the first sample (which is at 1, since the first sample is completely correlated with itself). This indicates that samples separated by 100 iterations are essentially uncorrelated for all practical purposes.

The progression from the full chain to l = 10 to l = 100 shows a consistent pattern: as the stride length increases, the autocorrelation decreases monotonically. With l = 100, even at lag 1 (which represents a separation of 100 original iterations), the correlation has effectively vanished. This is consistent with the expectation that thinning the chain reduces the dependence between retained samples, and the effect becomes more pronounced as the thinning interval increases.

For all three parameters (Φ, h, and σ_0^2), the ACF with l = 100 shows no significant correlation structure. Any remaining correlations are well within the confidence bands that would indicate statistical significance, meaning they are indistinguishable from zero given the finite sample size.

3.6

Based on the observed autocorrelation patterns, a stride length of $l \approx 50$ would be appropriate to obtain nearly-independent samples from the posterior distribution. This recommendation is supported by several considerations:

First, examining the ACF for the full chain, the autocorrelation decays to near-zero levels by around lag 30-40 for all parameters. This indicates that the underlying Markov chain has a relatively short correlation time, meaning that samples become effectively independent after approximately 30-40 iterations. A conservative choice of $l = 50$ ensures that we are safely beyond this correlation length.

Second, the progression from $l = 10$ to $l = 100$ shows diminishing returns. While $l = 10$ still shows some residual correlation (particularly at the first few lags), $l = 50$ would fall comfortably in the region where the ACF has already decayed to negligible levels. There is little practical benefit to using $l = 100$ over $l = 50$, as both produce effectively independent samples, but $l = 50$ retains twice as many samples for posterior inference.

Third, the choice of $l = 50$ balances the trade-off between sample independence and sample size. Thinning the chain too aggressively ($l = 100$ or higher) discards potentially useful information without meaningful improvement in independence, since the samples were already effectively independent at shorter lags. The goal of thinning is not to achieve perfect independence (which is impossible with finite samples), but rather to reduce autocorrelation to a level where it does not materially affect inference.

A confirmatory ACF plot with $l = 50$ should show that the autocorrelation at all positive lags is essentially zero, with any minor fluctuations falling well within the expected sampling variability. This confirms that samples retained every 50 iterations can be treated as approximately independent for the purposes of posterior summarization, credible interval construction, and density estimation.

```python
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat
from scipy.stats import norm, multivariate_normal
sys.path.append('../hw1')
import heat_rod as hr

def MetropolisHastingsMCMC(q0, J_func, r_calc, M, rng_seed=None):
    """
    General Metropolis-Hastings MCMC function

    Parameters:
    q0: initial sample
    J_func: proposal distribution function J(q* | q_{k-1})
    r_calc: function to compute acceptance ratio r(q*, q_{k-1})
    M: number of samples to generate
    rng_seed: random seed for reproducibility

    Returns:
    samples: array of MCMC samples
    acceptance_rate: acceptance rate of the chain
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, len(q0)))
    samples[0] = q_current

    accepted = 0

    print("Running Metropolis-Hastings MCMC...")
    for i in range(1, M):
        # Step 2.1: Generate proposal from J(q* | q_{k-1})
        q_proposed = J_func(q_current)

        # Step 2.2: Compute the ratio r(q*, q_{k-1})
        r = r_calc(q_proposed, q_current)

        # Step 2.3: Accept with probability min(1, r)
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        samples[i] = q_current

        if (i + 1) % 200 == 0:
            print(f"  Completed {i + 1}/{M} iterations, acceptance rate: {accepted/(i):.3f}")

    acceptance_rate = accepted / (M - 1)

    return samples, acceptance_rate

def setup_heat_equation_problem(mat_file='HW06_Problem1.mat'):
    """
    Set up the heat equation problem using data from MATLAB file
    """
    data = loadmat(mat_file)

    obs_data = data['observations'].flatten()
    print(f"Shape of observations: {obs_data.shape}")
    T_obs = obs_data

    # Observation locations (as given in the problem)
    x0 = 10   # cm
    dx = 4    # spacing
    x_obs = x0 + np.arange(len(obs_data)) * dx

    # Rod parameters
    a = 0.95  # cm
    b = 0.95  # cm
    k = 2.3   # W/cm/C
    Tamb = 21.29  # C
    L = 70    # cm

    q0 = np.array([-15.0, 0.002])  # [Phi, h]
    phi0, h0 = q0

    rod = hr.SteadyStateRod(a, b, phi0, k, h0, Tamb, L)

    # Known error variance
    sigma2_0 = 4.0  # (2 C)^2

    return rod, x_obs, T_obs, sigma2_0, q0

def create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0):
    """
    Create function to compute r(q*, q_{k-1}) = П‰(q*|Oμ) / П‰(q_{k-1}|Oμ)
    For Metropolis algorithm with symmetric proposal, this is the acceptance ratio
    """
    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    def posterior_ratio(q_star, q_prev):
        """
        Compute r(q*, q_{k-1}) = [ПЂ(П…|q*)ПЂ0(q*)] / [ПЂ(П…|q_{k-1})ПЂ0(q_{k-1})]
        """
        # Evaluate at q_star
        Phi_star, h_star = q_star
        rod.Phi_val = Phi_star
        rod.h_val = h_star
        T_model_star = rod.Ts(x_obs)

        # Log likelihood for q_star (using log to avoid underflow, then exponentiate)
        n = len(x_obs)
        log_likelihood_star = -0.5 * np.sum((T_obs - T_model_star)**2 / sigma2_0)

        # Log prior for q_star
        log_prior_star = (norm.logpdf(Phi_star, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_star, prior_h_mean, prior_h_std))

        # Evaluate at q_prev
        Phi_prev, h_prev = q_prev
        rod.Phi_val = Phi_prev
        rod.h_val = h_prev
        T_model_prev = rod.Ts(x_obs)

        # Log likelihood for q_prev
```

```
        log_likelihood_prev = -0.5 * np.sum((T_obs - T_model_prev)**2 / sigma2_0)

        # Log prior for q_prev
        log_prior_prev = (norm.logpdf(Phi_prev, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_prev, prior_h_mean, prior_h_std))

        # Log ratio
        log_ratio = (log_likelihood_star + log_prior_star) - (log_likelihood_prev + log_prior_prev)

        # Return actual ratio (exponentiate)
        return np.exp(log_ratio)

    return posterior_ratio

def run_metropolis_for_heat_equation(D =None, M = None, data_name = None):
    """
    Run Metropolis algorithm for the heat equation problem (Problem 1.1)
    """
    if data_name is None:
        data_name = 'HW06_Problem1.mat'

    # Setup problem
    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_name)
    if D is None:

        D = np.array([[1e-2, 0],
                      [0, 2e-10]])

    # Create proposal function J(q* | q_{k-1}) = N(q_{k-1}, D)
    def proposal_func(q_current):
        return np.random.multivariate_normal(q_current, D)

    # Create function to compute r(q*, q_{k-1})
    posterior_ratio_func = create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0)

    # Test at initial point
    initial_ratio = posterior_ratio_func(q0, q0)  # Should be 1.0
    print(f"Initial test - r(q0, q0) = {initial_ratio:.6f}")

    if M is None:
        M = 1000
    print(f"\nRunning Metropolis algorithm for {M} iterations...")
    print(f"Proposal distribution: N(q_prev, D) with D = diag([{D[0,0]}, {D[1,1]}])")

    samples, acceptance_rate = MetropolisHastingsMCMC(
        q0,
        proposal_func,
        posterior_ratio_func,
        M,
        rng_seed=42
    )

    print(f"\nFinal acceptance rate: {acceptance_rate:.3f}")

    # Find MAP estimate
    # We need to recompute the posterior (likelihood * prior) for each sample
    print("\nComputing posterior values for MAP estimate...")

    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    # Compute unnormalized posterior for each sample
    posterior_values = np.zeros(M)
    for i, q in enumerate(samples):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood
        n = len(x_obs)
        likelihood = np.exp(-0.5 * np.sum((T_obs - T_model)**2 / sigma2_0))

        # Prior
        prior = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                 norm.pdf(h, prior_h_mean, prior_h_std))

        # Unnormalized posterior (pi(data|q)pi0(q))
        posterior_values[i] = likelihood * prior

    # Find MAP estimate (maximum unnormalized posterior)
    map_idx = np.argmax(posterior_values)
    q_map = samples[map_idx]
    post_map = posterior_values[map_idx]

    print(f"\n{'='*60}")
    print("PROBLEM 1.1 RESULTS")
    print(f"{'='*60}")
    print(f"\nMAP Estimate (q_MAP):")
    print(f"  O¦ = {q_map[0]:.6f} W/cmBI")
    print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
    print(f"  ΠЪ(П…|q_MAP)ΠЪ0(q_MAP) = {post_map:.2e}")

    print(f"\nPosterior Distribution ΠЪ(q|П…) Summary:")
    print(f"{'='*60}")
    print(f"O¦:")
    print(f"  Mean = {np.mean(samples[:, 0]):.6f} W/cmBI")
    print(f"  Std  = {np.std(samples[:, 0]):.6f} W/cmBI")
    print(f"  95% CI = [{np.percentile(samples[:, 0], 2.5):.6f}, {np.percentile(samples[:, 0], 97.5):.6f}]")

    print(f"\nh:")
    print(f"  Mean = {np.mean(samples[:, 1]):.6e} W/cmBI/B°C")
    print(f"  Std  = {np.std(samples[:, 1]):.6e} W/cmBI/B°C")
    print(f"  95% CI = [{np.percentile(samples[:, 1], 2.5):.6e}, {np.percentile(samples[:, 1], 97.5):.6e}]")


    fig, axes = plt.subplots(2, 2, figsize=(14, 12))

    # Plot 1: Phi histogram (top left)
    axes[0, 0].hist(samples[:, 0], bins=30, density=True, alpha=0.7, color='skyblue', edgecolor='black')
    axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[0]:.4f}')
    axes[0, 0].set_xlabel('O¦ (W/cmBI)')
    axes[0, 0].set_ylabel('Density')
    axes[0, 0].set_title('Marginal Posterior ΠЪ(O¦|П…)')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    # Plot 2: h histogram (top right)
    axes[0, 1].hist(samples[:, 1], bins=30, density=True, alpha=0.7, color='lightcoral', edgecolor='black')
    axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[1]:.6f}')
```

```
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior ПЂ(h|П…)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Joint posterior scatter (bottom left)
axes[1, 0].scatter(samples[:, 0], samples[:, 1], alpha=0.5, c='purple', s=10)
axes[1, 0].plot(samples[:, 0], samples[:, 1], alpha=0.5, c='purple')
axes[1, 0].scatter(q_map[0], q_map[1], color='Magenta', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O¦ (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples ПЂ(O¦,h|П…)')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Model fit with MAP parameters (bottom right)
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('problem1.1_results.png', dpi=150)
plt.show()
return samples, q_map, posterior_values, acceptance_rate
```

```python
from mcmc import *
from dram import *
def adaptive_metropolis_mcmc(q0, J_func, r_calc, M, rng_seed=None):
    """
    Metropolis-Hastings MCMC with Adaptive Metropolis (AM)

    Parameters:
    q0: initial sample
    J_func: initial proposal distribution function (for first 100 iterations)
    r_calc: function to compute acceptance ratio r(q*, q_{k-1})
    M: number of samples to generate
    rng_seed: random seed for reproducibility

    Returns:
    samples: array of MCMC samples
    acceptance_rate: acceptance rate of the chain
    Vk_at_101: the covariance matrix computed at k=101 (for problem 2.1)
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0)  # dimension of parameter space (should be 2)
    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, p))
    samples[0] = q_current

    # AM parameters
    sp = 2.38**2 / p  # scaling factor (2.38^2/p)
    k0 = 100  # start adapting after 100 samples

    accepted = 0
    Vk_at_101 = None  # to store the covariance at k=101 for problem 2.1

    print("Running Adaptive Metropolis MCMC...")
    for i in range(1, M):
        # Step 2.1: Generate proposal
        if i < k0:
            # First k0 iterations: use fixed proposal J_func
            q_proposed = J_func(q_current)
        else:
            # Adaptive Metropolis proposal
            if i == k0:
                # First time using AM (k=101)
                # Compute covariance from samples[0:k0] (first 100 samples)
                samples_so_far = samples[:k0]
                Vk = np.cov(samples_so_far, rowvar=False)
                Vk_at_101 = Vk.copy()  # Save for problem 2.1
                print(f"\nAt k=101, computed Vk from first {k0} samples:")
                print(f"Vk = {Vk}")

                # Use this Vk for the proposal at k=101
                proposal_cov = sp * Vk
            elif np.mod(i, k0)==1:
                # For subsequent iterations, compute covariance from all samples up to i
                samples_so_far = samples[:i]
                Vk = np.cov(samples_so_far, rowvar=False)
                proposal_cov = sp * Vk

            # Generate proposal from N(q_current, sp * Vk)
            q_proposed = np.random.multivariate_normal(q_current, proposal_cov)

        # Step 2.2: Compute the ratio r(q*, q {k-1})
        r = r_calc(q_proposed, q_current)

        # Step 2.3: Accept with probability min(1, r)
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        samples[i] = q_current

        if (i + 1) % 100 == 0:
            print(f"  Completed {i + 1}/{M} iterations, acceptance rate: {accepted/(i):.3f}")

    acceptance_rate = accepted / (M - 1)

    return samples, acceptance_rate, Vk_at_101, Vk

def run_adaptive_metropolis_for_heat_equation(M=None, func = None, k_for_am = None, burn = None, title = 'AM', data_filename = 'HW06_Problem1.mat'):
    """
    Run Adaptive Metropolis algorithm for the heat equation problem (Problem 1.1)
    """
    # Setup problem
    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_filename)

    # Initial proposal covariance (same as in Problem 1.1)
    D = np.array([[1e-2, 0],
                  [0, 2e-10]])

    # Create initial proposal function J(q* | q_{k-1}) = N(q_{k-1}, D)
    # This will be used for the first 100 iterations
    def initial_proposal_func(q_current, returnd=False):
        return (
            np.random.multivariate_normal(q_current, D),
            D
        ) if returnd else np.random.multivariate_normal(q_current, D)
    # Create function to compute r(q*, q_{k-1})
    posterior_ratio_func = create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0)

    if M is None:
        M = 1000

    print(f"\nRunning Adaptive Metropolis algorithm for {M} iterations...")
    print(f"Initial proposal distribution (first {100} iterations): N(q_prev, D) with D = diag([{D[0,0]}, {D[1,1]}])")
    print(f"AM parameters: sp = 2.38^2/{len(q0)} = {2.38**2/len(q0):.4f}, k0 = 100")
    if func == 'dram':
        if data_filename is None:
            data_filename = 'HW06_Problem3.mat'

        rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_filename)
        ns=0.01; sigma2_s=4.0

        samples, acceptance_rate, Vk_at_101, Vk_at_end = dram(
        q0 = q0, s2_0 = sigma2_0,
        J_func = initial_proposal_func,
        r_calc = posterior_ratio_func,
        M =  M, ns = ns, sigma2_s = sigma2_s, n_obs = len(T_obs), rod = rod,  x_obs =x_obs, T_obs = T_obs,
        rng_seed=42, k = k_for_am
    )
        print("am,san", samples.shape)


    elif func == 'drm':
```

```
    samples, acceptance_rate, Vk_at_101, Vk_at_end = drm(
    q0,
    initial_proposal_func,
    posterior_ratio_func,
    M,
    rng_seed=42
)
else:
    samples, acceptance_rate, Vk_at_101, Vk_at_end = adaptive_metropolis_mcmc(
        q0,
        initial_proposal_func,
        posterior_ratio_func,
        M,
        rng_seed=42
    )

print(f"\nFinal acceptance rate: {acceptance_rate:.3f}")

# Problem 2.1: Report Vk at k=101
print(f"\n{'='*60}")
print("PROBLEM 2.1 RESULTS")
print(f"{'='*60}")
print(f"\nAt k=101, the new Vk generated from the first 100 samples is:")
print(f"Vk = {Vk_at_101}")

# Problem 2.2: Report Vk at final k
print(f"\n{'='*60}")
print("PROBLEM 2.2 RESULTS")
print(f"{'='*60}")
print(f"\nAt k=M, the final Vk generated is:")
print(f"Vk = {Vk_at_end}")


print("\nComputing posterior values for MAP estimate...")

# Prior parameters
prior_Phi_mean = -15
prior_Phi_std = 10
prior_h_mean = 0.001
prior_h_std = 0.005

# Compute unnormalized posterior for each sample
print("\nComputing posterior values for MAP estimate...")
posterior_values = np.zeros(M)
if func == "dram": # unknwon sigma**2
      # Find MAP estimate (maximum unnormalized posterior with sigma_squared)

    print("SHAPE", samples.shape)
    q_samples, s2_samples = samples[:,:-1], samples[:,-1]
    q_samples_post = q_samples[burn:]
    s2_samples_post = s2_samples[burn:]
    posterior_values = np.zeros(len(q_samples_post))
    for i, (q, s2) in enumerate(zip(q_samples_post, s2_samples_post)):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood with current sigma_squared
        n = len(x_obs)
        likelihood = (1/np.sqrt(2*np.pi*s2))**n * np.exp(-0.5 * np.sum((T_obs - T_model)**2 / s2))

        # Prior for q
        prior_q = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                norm.pdf(h, prior_h_mean, prior_h_std))

        # Prior for sigma_squared (Inverse Gamma)
        # p(sigma_squared) \propto (sigma_squared)^(-(ns/2+1)) * exp(-(ns*sigma_squared_s)/(2sigma_squared))
        log_prior_sigma = -(ns/2 + 1) * np.log(s2) - (ns * sigma2_s) / (2 * s2)
        prior_sigma = np.exp(log_prior_sigma)

        # Unnormalized posterior
        posterior_values[i] = likelihood * prior_q * prior_sigma


        map_idx = np.argmax(posterior_values)
        q_map = q_samples_post[map_idx]
        s2_map = s2_samples_post[map_idx]
        post_map = posterior_values[map_idx]

else:
    for i, q in enumerate(samples):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood
        n = len(x_obs)
        likelihood = np.exp(-0.5 * np.sum((T_obs - T_model)**2 / sigma2_0))

        # Prior
        prior = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                norm.pdf(h, prior_h_mean, prior_h_std))

        # Unnormalized posterior (pi(data|q)pi0(q))
        posterior_values[i] = likelihood * prior

    # Find MAP estimate (maximum unnormalized posterior)
    map_idx = np.argmax(posterior_values)
    q_map = samples[map_idx]
    post_map = posterior_values[map_idx]

print(f"\n{'='*60}")
print("PROBLEM 1.1 RESULTS")
print(f"{'='*60}")
print(f"\nMAP Estimate (q_MAP):")
print(f"  O¦ = {q_map[0]:.6f} W/cmBI")
print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
print(f"  ПЂ(П…|q_MAP)ПЂ0(q_MAP) = {post_map:.2e}")

print(f"\nPosterior Distribution ПЂ(q|П…) Summary:")
print(f"{'='*60}")
print(f"O¦:")
print(f"  Mean = {np.mean(samples[:, 0]):.6f} W/cmBI")
print(f"  Std  = {np.std(samples[:, 0]):.6f} W/cmBI")
print(f"  95% CI = [{np.percentile(samples[:, 0], 2.5):.6f}, {np.percentile(samples[:, 0], 97.5):.6f}]")

print(f"\nh:")
print(f"  Mean = {np.mean(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  Std  = {np.std(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  95% CI = [{np.percentile(samples[:, 1], 2.5):.6e}, {np.percentile(samples[:, 1], 97.5):.6e}]")
```

```
# Create diagnostic plots - 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# Plot 1: Phi histogram (top left)
axes[0, 0].hist(samples[:, 0], bins=30, density=True, alpha=0.7, color='skyblue', edgecolor='black')
axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[0]:.4f}')
axes[0, 0].set_xlabel('O¦ (W/cmBI)')
axes[0, 0].set_ylabel('Density')
axes[0, 0].set_title('Marginal Posterior ПЂ(O¦|П…)')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: h histogram (top right)
axes[0, 1].hist(samples[:, 1], bins=30, density=True, alpha=0.7, color='lightcoral', edgecolor='black')
axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[1]:.6f}')
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior ПЂ(h|П…)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Joint posterior scatter (bottom left)
axes[1, 0].scatter(samples[:, 0], samples[:, 1], alpha=0.5, c='purple', s=10)
axes[1, 0].plot(samples[:, 0], samples[:, 1], alpha=0.5, c='purple')
axes[1, 0].scatter(q_map[0], q_map[1], color='Magenta', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O¦ (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples ПЂ(O¦,h|П…)')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Model fit with MAP parameters (bottom right)
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
p = f'{title}.png'
os.path.exists(p)
plt.savefig(p, dpi=150)
plt.show()

return samples, posterior_values, q_map, Vk_at_101, acceptance_rate
```

```python
import sys, os, numpy as np
from am_mcmc import *

def SSq_func(rod, q_current, x_obs, T_obs):
        Phi, h = q_current
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)
        SSq = np.sum((T_obs - T_model)**2)
        return SSq

def gibbs_sigma2(q_current, ns, sigma2_s, n_obs, rod, x_obs, T_obs):
    SSq = SSq_func(rod, q_current, x_obs, T_obs)

    aval = ns/2 + n_obs/2
    bval = (ns * sigma2_s)/2 + SSq/2

    gamma_sample = np.random.gamma(aval, 1/bval)
    return 1 / gamma_sample

def dram(q0, s2_0, J_func,  r_calc, M, ns, sigma2_s, n_obs, rod,  x_obs, T_obs, SSq_func = SSq_func,
         rng_seed=None, k=None):

    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0)
    q_current = np.array(q0, dtype=float)
    s2_current = s2_0
    samples = np.zeros((M, p + 1))
    samples[0, :p] = q_current
    samples[0, p] = s2_current

    sp = 2.38**2 / p
    gamma2 = 1/25   # delayed rejection cov scaling

    k0 = k

    accepted = 0
    Vk_at_101 = None
    Vk = np.zeros(p)
    for i in range(1, M):

        # ----------------------------
        # Compute AM covariance
        # ----------------------------
        if i < k0:
            q_proposed,D = J_func(q_current, True)

            proposal_cov = D
        else:
            if i == k0:
                Vk = np.cov(samples[:k0, :p], rowvar=False)
                Vk_at_101 = Vk.copy()
            elif np.mod(i, k0) == 1:
                Vk = np.cov(samples[:i, :p], rowvar=False)

            proposal_cov = sp * Vk
            q_proposed = np.random.multivariate_normal(q_current, proposal_cov)

        r_calc = create_posterior_ratio_func(rod, x_obs, T_obs, s2_current)
        # ----------------------------
        # Stage 1 acceptance
        # ----------------------------
        # r_calc = create_posterior_ratio_func(rod, x_obs, T_obs, s2_current)
        # r1 = r_calc(q_proposed, q_current) # dont need sigma, since J is gaussian (it cancels in r( qstar , q^k-1))
        r1 = r_calc(q_proposed, q_current)
        alpha1 = min(1, r1)

        if np.random.rand() < alpha1:
            q_current = q_proposed
            accepted += 1

        else:
            # ----------------------------
            # Delayed Rejection Stage 2
            # ----------------------------

            # if i > k0:
            proposal_cov2 = gamma2**2 * proposal_cov
            q_proposed2 = np.random.multivariate_normal(q_current, proposal_cov2)

            r2 = r_calc(q_proposed2, q_current)

            # Need alpha1(q_proposed2, q_proposed)
            r1_reverse = r_calc(q_proposed, q_proposed2)
            alpha1_reverse = min(1, r1_reverse)

            numerator = r2 * (1 - alpha1_reverse)
            denominator = (1 - alpha1)

            alpha2 = min(1, numerator / denominator) if denominator > 0 else 1

            if np.random.rand() < alpha2:
                q_current = q_proposed2
                accepted += 1

        # Gibbs update for sigma^2
        s2_current = gibbs_sigma2(q_current, ns, sigma2_s, n_obs, rod, x_obs, T_obs)

        samples[i, :p] = q_current
        samples[i, p] = s2_current

    acceptance_rate = accepted / (M - 1)
    print("sapesle", samples.shape)
    return samples, acceptance_rate, Vk_at_101, Vk


def drm(q0, J_func, r_calc, M, rng_seed=None):
    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0)
    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, p))
    samples[0] = q_current

    sp = 2.38**2 / p ## only for AM
    gamma2 = 1/25   # delayed rejection cov scaling



    accepted = 0
```

```
Vk_at_101 = None
Vk = np.zeros(p)
for i in range(1, M):


    q_proposed, proposal_cov = J_func(q_current, returnd = True)

    # ----------------------------
    # Stage 1 acceptance
    # ----------------------------
    r1 = r_calc(q_proposed, q_current)
    alpha1 = min(1, r1)

    if np.random.rand() < alpha1:
        q_current = q_proposed
        accepted += 1

    else:
        # ----------------------------
        # Delayed Rejection Stage 2
        # ----------------------------
        proposal_cov2 = gamma2 * proposal_cov
        q_proposed2 = np.random.multivariate_normal(q_current, proposal_cov2)

        r2 = r_calc(q_proposed2, q_current)

        # Need alpha1(q_proposed2, q_proposed)
        r1_reverse = r_calc(q_proposed, q_proposed2)
        alpha1_reverse = min(1, r1_reverse)

        numerator = r2 * (1 - alpha1_reverse)
        denominator = (1 - alpha1)

        alpha2 = min(1, numerator / denominator) if denominator > 0 else 1

        if np.random.rand() < alpha2:
            q_current = q_proposed2
            accepted += 1

    samples[i] = q_current

acceptance_rate = accepted / (M - 1)

return samples, acceptance_rate, Vk_at_101, Vk
```